

요구사항 분석서

GitHub Repository: <https://github.com/CodingHwangga/bandmates>

프로젝트명: BandMate

학번: 2022125069 | 이름: 황주원

작성일: 2026.05.15

목차

- 서론
- 시스템 개요
- 요구사항 분석 및 명세
- 인터페이스 분석
- 제약사항 및 요구사항 추적
- 참고문헌 및 부록

1. 서론

1.1 문서의 목적 및 범위

본 문서는 BandMate 시스템의 요구사항을 분석하고, 시스템의 구조와 동작을 명세하는 것을 목적으로 한다.

문서의 범위:

- 시스템 문맥도 및 기능 분류
- 정적/동적 분석
- 객체 및 행위 분석
- 인터페이스 명세
- 요구사항 추적

1.2 용어 정의

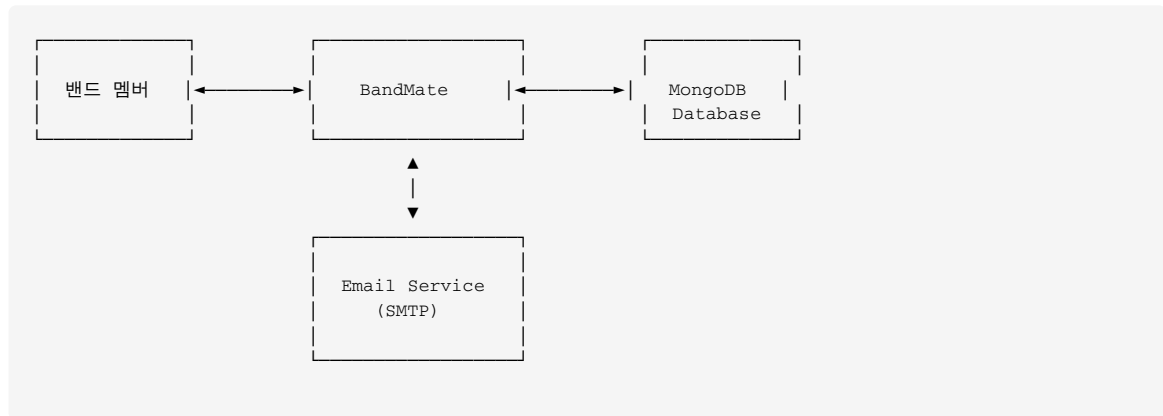
용어	설명
사용자	BandMate 시스템을 사용하는 밴드 멤버
밴드 리더	밴드를 생성하고 관리하는 사용자
세션	사용자의 로그인 상태
CRUD	Create, Read, Update, Delete 기본 연산

1.3 참조 문서

- 과제1. 프로젝트정의서
- 과제2. 프로젝트관리계획서
- 과제3. 요구사항정의서
- GitHub Repository: <https://github.com/CodingHwangga/bandmates>

2. 시스템 개요

2.1 소프트웨어 문맥도



외부 시스템:

- **MongoDB:** 데이터 저장소
- **Email Service:** 멤버 초대 이메일 발송

2.2 주요 기능 분류

2.2.1 사용자 관리 기능

- 회원가입/로그인
- 프로필 관리
- 인증 및 세션 관리

2.2.2 밴드 관리 기능

- 밴드 CRUD
- 멤버 초대 및 관리
- 파트(역할) 지정

2.2.3 일정 관리 기능

- 합주 일정 CRUD
- 캘린더 뷰
- 일정 알림

2.2.4 콘텐츠 관리 기능

- 연습곡 관리
- 셋리스트 구성
- 파일 업로드/다운로드
- 녹음 파일 관리

3. 요구사항 분석 및 명세

3.1 정적 분석

3.1.1 주요 엔티티 (Entity)

User (사용자)

- 속성: userId, email, password, name, profileImage
- 설명: 시스템을 사용하는 사용자

Band (밴드)

- 속성: bandId, name, genre, description, createdAt
- 설명: 사용자들이 속한 밴드 그룹

Member (멤버)

- 속성: memberId, userId, bandId, role, part
- 설명: 밴드에 속한 사용자의 역할 정보

Schedule (일정)

- 속성: scheduleId, bandId, title, date, time, location
- 설명: 밴드의 합주 일정

Song (곡)

- 속성: songId, bandId, title, artist, genre, status
- 설명: 밴드가 연습하는 곡

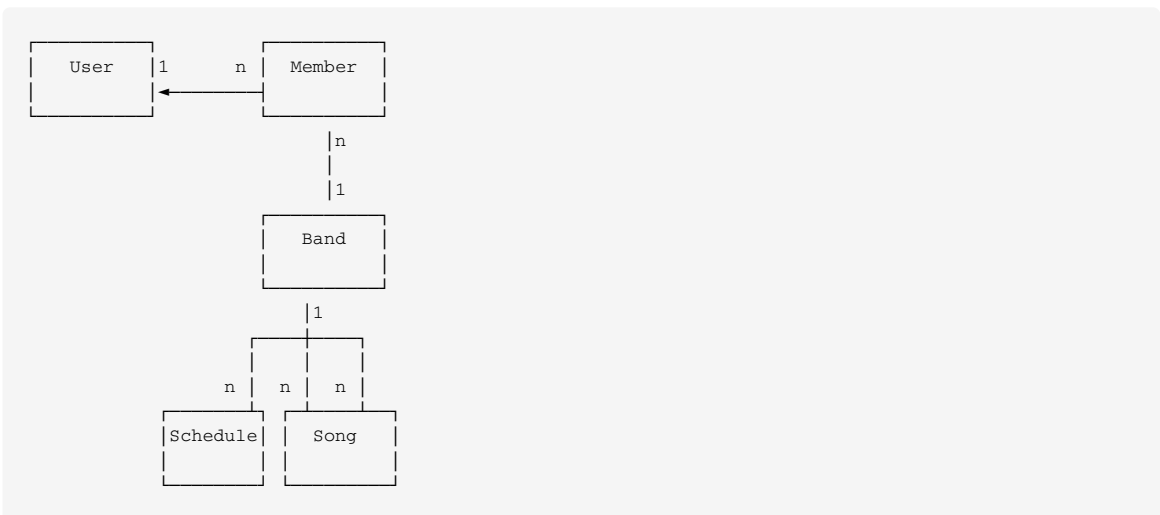
File (파일)

- 속성: fileId, songId, fileName, fileType, uploadDate, url
- 설명: 악보, MR 등의 파일

Recording (녹음)

- 속성: recordingId, bandId, scheduleId, fileName, memo, recordDate
- 설명: 합주 녹음 파일

3.1.2 ER 다이어그램

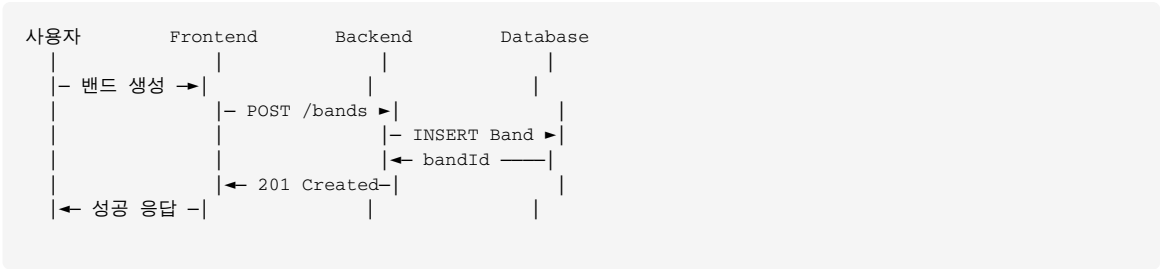


| 1
|
| n

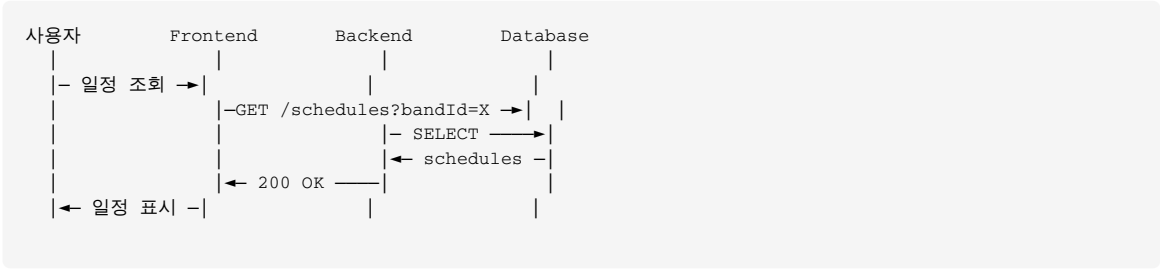
File

3.2 동적 분석

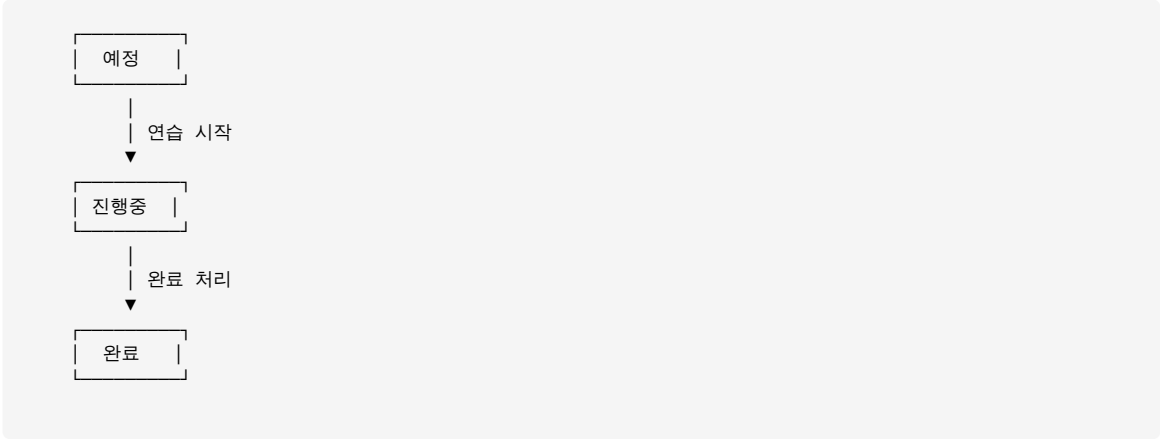
3.2.1 시퀀스 다이어그램 - 밴드 생성



3.2.2 시퀀스 다이어그램 - 일정 조회



3.2.3 상태 다이어그램 - 곡 연습 상태



3.3 객체 및 행위 분석

3.3.1 주요 클래스

BandController

- 책임: 밴드 관련 요청 처리
- 주요 메서드:
 - createBand(bandData)
 - getBandInfo(bandId)
 - inviteMember(bandId, email)
 - updateBand(bandId, bandData)

ScheduleController

- 책임: 일정 관리
- 주요 메서드:
 - createSchedule(scheduleData)
 - getSchedules(bandId, month)
 - updateSchedule(scheduleId, scheduleData)
 - deleteSchedule(scheduleId)

SongController

- 책임: 곡 및 셋리스트 관리
- 주요 메서드:
 - addSong(songData)
 - getSongs(bandId)
 - updateSongStatus(songId, status)
 - createSetlist(bandId, songIds)

FileController

- 책임: 파일 업로드/다운로드
- 주요 메서드:
 - uploadFile(file, songId)
 - downloadFile(fileId)
 - deleteFile(fileId)

3.3.2 유스케이스 다이어그램



녹음 재생

4. 인터페이스 분석

4.1 사용자 인터페이스

4.1.1 주요 화면 구성

메인 대시보드

- 내 밴드 목록
- 다가오는 일정
- 최근 활동

밴드 상세 페이지

- 밴드 정보
- 멤버 목록
- 탭 메뉴 (일정/곡/파일/녹음)

캘린더 페이지

- 월간/주간 뷰
- 일정 생성 모달
- 일정 상세 정보

곡 관리 페이지

- 곡 목록 테이블
- 셋리스트 구성
- 파일 업로드 영역

4.1.2 화면 흐름도



4.2 시스템 간 인터페이스

4.2.1 REST API 명세

밴드 관리 API

Method	Endpoint	설명	요청	응답
POST	/api/bands	밴드 생성	{name, genre, description}	{bandId, ...}
GET	/api/bands/:id	밴드 조회	-	{band 정보}
POST	/api/bands/:id/invite	멤버 초대	{email}	{success: true}
GET	/api/bands/:id/members	멤버 목록	-	[{member}]

일정 관리 API

--	--	--	--	--

Method	Endpoint	설명	요청	응답
POST	/api/schedules	일정 생성	{bandId, title, date, time, location}	{scheduleId, ...}
GET	/api/schedules	일정 조회	?bandId=X&month=Y	[{schedule}]
PUT	/api/schedules/:id	일정 수정	{title, date, time, location}	{updated}
DELETE	/api/schedules/:id	일정 삭제	-	{deleted: true}

곡 관리 API

Method	Endpoint	설명	요청	응답
POST	/api/songs	곡 등록	{bandId, title, artist, genre}	{songId, ...}
GET	/api/songs	곡 목록	?bandId=X	[{song}]
PUT	/api/songs/:id/status	상태 변경	{status}	{updated}

4.3 입력/출력 인터페이스

4.3.1 입력 데이터

밴드 생성 입력

```
{
  "name": "The Beatles Cover",
  "genre": "Rock",
  "description": "비틀즈 커버 밴드"
}
```

일정 등록 입력

```
{
  "bandId": "band123",
  "title": "정기 합주",
  "date": "2026-05-20",
  "time": "19:00",
  "location": "홍대 연습실 A"
}
```

4.3.2 출력 데이터

밴드 정보 출력

```
{
  "bandId": "band123",
  "name": "The Beatles Cover",
  "genre": "Rock",
  "description": "비틀즈 커버 밴드",
  "createdAt": "2026-04-01",
  "members": [
    {
      "userId": "user1",
      "name": "홍길동",
      "part": "기타"
    }
  ]
}
```

5. 제약사항 및 요구사항 추적

5.1 기술적 제약사항

ID	제약사항	영향
TC-01	무료 호스팅 플랫폼 사용	성능 제한, 동시 접속자 제한
TC-02	MongoDB 무료 플랜 (512MB)	저장 용량 제한
TC-03	파일 크기 제한 (50MB)	대용량 파일 업로드 불가
TC-04	Node.js v18 이상 필요	개발 환경 제약

5.2 운영적 제약사항

ID	제약사항	대응 방안
OC-01	1인 개발	우선순위 기능 먼저 구현
OC-02	12주 개발 기간	MVP 중심 개발
OC-03	테스트 환경 제한	로컬 테스트 위주

5.3 요구사항 추적표

요구사항 ID	분석 산출물	설계 산출물	구현 예정	테스트 계획
FR-U-01~03	User Entity	UserController	Week 6	단위 테스트
FR-B-01~05	Band Entity	BandController	Week 6	통합 테스트
FR-S-01~04	Schedule Entity	ScheduleController	Week 7	통합 테스트
FR-M-01~05	Song Entity	SongController	Week 7	단위 테스트
FR-F-01~04	File Entity	FileController	Week 8	통합 테스트
FR-R-01~04	Recording Entity	RecordingController	Week 8	단위 테스트

5.4 요구사항 검증 매트릭스

요구사항	검증 방법	합격 기준
FR-U-01	회원가입 테스트	정상 등록 확인
FR-B-01	밴드 생성 테스트	DB에 저장 확인
FR-S-03	캘린더 렌더링	화면 표시 확인
FR-F-01	파일 업로드 테스트	파일 저장 확인
NFR-P-01	성능 테스트	3초 이내 응답
NFR-S-01	보안 테스트	암호화 확인

6. 참고문헌 및 부록

6.1 참고문헌

- 과제1. 프로젝트정의서
- 과제2. 프로젝트관리계획서
- 과제3. 요구사항정의서
- UML 표기법 가이드
- REST API 설계 가이드

6.2 부록

부록 A. 데이터 모델 상세

User 스키마

```
{
  userId: String (Primary Key),
  email: String (Unique),
  password: String (Hashed),
  name: String,
  profileImage: String,
  createdAt: Date
}
```

Band 스키마

```
{
  bandId: String (Primary Key),
  name: String,
  genre: String,
  description: String,
  createdAt: Date
}
```

부록 B. API 응답 코드

코드	의미	사용 사례
200	OK	조회 성공
201	Created	생성 성공
400	Bad Request	잘못된 요청
401	Unauthorized	인증 실패
404	Not Found	리소스 없음
500	Server Error	서버 오류

작성자: 황주원

학번: 2022125069

버전: v1.0

최종 수정일: 2026.05.15